Comparing TurboQuant+ with traditional quantization methods on multiple LLM

Stanford CS224N {Custom, Default} Project

Matias Soto Carvajal
Faculty of Computer Science
Ruhr-Universität Bochum
Matias.SotoCarvajal@edu.ruhr-uni-bochum.de

Name 2
Faculty of Computer Science
Ruhr-Universität Bochum
name@stanford.edu

Name 3
Faculty of Computer Science
Ruhr-Universität Bochum
name@stanford.edu

Abstract

An abstract should concisely (less than 300 words) motivate the problem, describe your aims, describe your contribution, and highlight your main finding(s).

1 Key Information to include

- Mentor:
- External Collaborators (if you have any):
- Sharing project:

2 Introduction

The introduction explains the problem, why it's difficult, interesting, or important, how and why current methods succeed/fail at the problem, and explains the key ideas of your approach and results. Though an introduction covers similar material as an abstract, the introduction gives more space for motivation, detail, references to existing work, and to capture the reader's interest.

Today, Large Language Models (LLMs) are highly advanced. They are capable of performing highly complex analysis and comprehension tasks across various domains. Furthermore, thanks to AI agents, they can write code, manipulate files within an operating system, and even control a computer using multiple tools available to the agent. These latter tasks, being highly complex, require storing and processing a massive amount of tokens during inference, as well as preserving the conversation context to maintain coherence throughout an AI agent's session. To achieve this, models utilize a technique called Key-Value (KV) Cache, which stores the mathematical representations of previously processed tokens. By doing so, instead of reprocessing all past tokens every time a new token is generated, the model simply retrieves their mathematical representations from the device's VRAM. Using this cache to generate new tokens significantly saves both memory space and computation time. With this technique applied, models are able to support much longer context windows and maintain coherence during a session or conversation.

The major drawback of storing this cache in VRAM is that, once a large number of tokens have been processed during an LLM session, the cache can grow to several gigabytes. It can even exceed the memory footprint of the model weights themselves. This means that for current complex tasks and AI agents, a substantial amount of VRAM must be provisioned just for the cache, in addition to loading the model.

Multiple solutions have been proposed for this problem. The most common approach is to quantize the KV Cache, reducing the size of the information without significantly degrading the quality of the model's responses. The simplest of these solutions involves quantizing the Key and Value matrices to lower bit levels. Dropping from 16 bits to 8 bits provides significant memory gains with minimal quality loss. However, below 8 bits, models begin to experience a very noticeable degradation in response quality. At extremely low bit levels, the model's syntax generation starts to break down, causing features like Tool Calling to fail frequently. Currently, the most widely used method is quantizing the KV Cache to Q8, as it significantly reduces memory usage while having a minimal impact on model quality.

To address this problem, Google released their paper on TurboQuant on March 24, 2026. This is an advanced memory optimization method.

2.1 TurboQuant

TurboQuant is a method that introduces several novel features, making it theoretically much more effective than simpler solutions:

1. It allows for dynamic memory compression during execution.
2. It applies PolarQuant: it dynamically rotates memory vectors and applies a transformation from Cartesian to polar coordinates. This transformation enables even further memory compression.
3. It uses a single-bit QJL correction technique to mitigate the quality loss incurred in the previous steps.

During their experiments with this technique, the researchers realized something crucial: **V is free, K is everything.** It is only possible to quantize K up to a reasonable limit (such as Q8), whereas V tolerates highly aggressive compression with almost no loss in response quality. This is a significant revelation because, alongside the aforementioned transformations, it enables asynchronous quantization in models by combining different compression methods for K and V. This allows the search for an ideal pair that maximizes memory savings while minimizing information loss.

This is the primary focus of the current project. Different combinations of KV Cache quantization will be tested across various LLMs, and the results will be analyzed to verify whether there is indeed a notable improvement when using this technique.

3 Related Work

This section helps the reader understand the research context of your work by providing an overview of existing work in the area.

4 Approach

This section details your approach(es) to the problem. For example, this is where you describe the architecture of your neural network(s), and any other key methods or algorithms.

To experiment and determine whether there is any real benefit to applying TurboQuant quantizations, it is necessary to establish a solid baseline. The theoretical benefits of TurboQuant start to become apparent in memory compression during long-context workloads.

4.1 Models

The following open-weight models were used to run the benchmarks:

1. Gemma 4 (E4B and E2B) Q8
2. Qwen 3.5 9B Q8
3. Llama 3.1 8B Q8

All of these models were downloaded directly from HuggingFace. These models were selected because they are among the most popular within the open-weight community, offer a good quality-to-size ratio, and are the most compatible with the TurboQuant fork.

4.2 Framework

We used the Llama-cpp-turboquant fork. This fork adds the TurboQuant codec stack to Llama.cpp, allowing for the quantization of both the model weights and the KV Cache. Furthermore, being a direct fork of Llama.cpp, it inherits all of its advantages, such as compatibility with various inference backends (CUDA, Vulkan, Metal), the ability to start LLM servers, and support for multiple models from different providers.

4.3 Work Environment

All tests were executed on a Linux instance rented from Vast.ai, providing access to an RTX 5090 32GB delivering approximately 100 TFLOPS. This allowed us to increase the test sample size to obtain more precise results. Unfortunately, the Llama.cpp-turboquant fork does not provide pre-built binaries, so it was necessary to build them directly within the container to use the most critical binaries for testing. The connection was established via SSH, allowing us to connect internally to the instance, clone the repository, and execute the tests.

5 Experiments

This section contains the following.

The tests conducted in this project were as follows:

5.1 Data

Describe the dataset(s) you are using (provide references). If it's not already clear, make sure the associated task is clearly described. Being precise about the exact form of the input and output can be very useful for readers attempting to understand your work, especially if you've defined your own task.

The dataset used to run the benchmark was LongBench V2 by zai-org. This dataset consists of 503 multiple-choice problems covering a variety of topics, context sizes, and difficulty levels, making it a highly demanding challenge for LLMs. To optimize for time and the cost of the rented GPU, LongBench was utilized as follows:

- 50 random samples were selected. These 50 tasks remain identical across all models and KV Cache quantization pairs.
- These samples range from short to medium and large tasks.
- Short and medium tasks were prioritized due to the defined context window for the models.

5.2 Evaluation method

Describe the evaluation metric(s) you use, plus any other details necessary to understand your evaluation. Some projects will have clear metrics from prior work on given datasets, but we realize that other projects will define their own metrics. If you're defining your own metrics, be clear as to what you're hoping to measure with each evaluation method (whether quantitative or qualitative, automatic or human-defined!), and how it's defined.

Multiple tests were designed for the models and their KV Cache pairs to broadly verify the overall performance of these quantizations.

5.2.1 LongBench V2

A Python script was designed to run the tests automatically across all models, following this sequence:

1. The 50 samples are selected.

Table 1: Idle GPU-memory allocation with traditional and Turbo4 KV caches.

Model	Traditional	Turbo4	Saving	Reduction
Qwen3.5-9B	10,310 MiB	9,662 MiB	648 MiB	6.29%
Llama 3.1 8B	12,900 MiB	10,672 MiB	2,228 MiB	17.27%

2. The model is loaded with its weights (Q8_0) using Llama-server.
3. The quantization type for the K and V values is defined in Llama-server. The pairs selected for testing are:
   (a) K: F16 V: F16 (Full precision)
   (b) K: Q8_0 V: Q8_0 (Most common quantization)T
   (c) K: Q8_0 V: turbo3 (Good balance between memory savings and quality)
   (d) K: turbo4 V: Q8_0 (More experimental and unstable combination)
   (e) K: turbo4 V: turbo4 (Most aggressive and delicate quantization)
4. The 50 tasks are executed on each KV Cache pair combination for every model.
5. Once the 50 tasks are completed, important metrics are recorded (Number of correct answers out of 50, TTFT, Decode and Encode speed, Peak VRAM Usage, etc.).
6. The process is repeated from step two for the remaining models.

5.2.2 Ruler Tests

Fill this

5.2.3 Hermes Agent

We evaluated Qwen3.5-9B and Meta-Llama-3.1-8B-Instruct on an NVIDIA GeForce RTX 5090 using a 65,536-token context window, full GPU offloading, flash attention, and Hermes Agent. Both configurations used the same Q8_0 model weights. The controlled variable within each model pair was the KV-cache format: the traditional configuration used Q8_0 for both the key and value caches, whereas the TurboQuant configuration used Turbo4 for both caches. Consequently, these experiments evaluate Turbo4 KV-cache compression rather than TurboQuant weight quantization.

5.3 Experimental details

Report how you ran your experiments (e.g., model configurations, learning rate, training time, etc.)

5.4 Results

Report the quantitative results that you have found. Use a table or plot to compare results and compare against baselines.

- If you're a default project team, you should **report the accuracy and Pearson correlation scores you obtained on the test leaderboard** in this section. You can also report dev set results if you'd like.
- Comment on your quantitative results. Are they what you expected? Better than you expected? Worse than you expected? Why do you think that is? What does that tell you about your approach?

5.4.1 Hermes Agent: Qwen and Llama KV-Cache Comparison

GPU-memory results Turbo4 reduced idle GPU-memory allocation for both models. Qwen3.5-9B saved 648 MiB, corresponding to a 6.29% reduction, while Llama 3.1 8B saved 2,228 MiB, corresponding to a 17.27% reduction. The larger reduction for Llama indicates that the benefit of KV-cache compression depends on the model architecture and cache dimensions. This is the clearest improvement demonstrated by the experiments.

Table 2: Summary of the autonomous travel-planning results.

Configuration	File generation	Main correctness problem	Result
Qwen traditional	9/9 files	Final cost was €9,290	Failed
Qwen Turbo4	9/9 files	Detailed cost was about €6,947.50	Failed
Llama traditional	1/9 files	Paris-only itinerary and unsupported budget	Failed
Llama Turbo4	Claimed complete	Minimum stated cost was €5,800	Failed

Runtime, utilization, and power For the Qwen travel-planning task, the traditional configuration completed in 2,199.93 seconds, whereas Turbo4 required 2,269.75 seconds. Turbo4 was therefore approximately 69.82 seconds, or 3.17%, slower in this run. Average GPU utilization was similar at 92.50% and 93.33%, respectively. Average power also remained effectively unchanged, decreasing from 563.19 W to 561.48 W. Because the Turbo4 run lasted longer, its estimated energy consumption increased slightly from approximately 0.344 kWh to 0.353 kWh.

For Llama, both configurations repeatedly reached 99–100% GPU utilization and consumed approximately 575 W during active inference. However, the available records did not contain sufficiently consistent execution-time, average-power, or energy measurements for a controlled Llama speed comparison. Therefore, no speed improvement can be claimed for Llama.

The Qwen document-analysis run also does not demonstrate a raw inference speed advantage. The traditional run required approximately 884 seconds, while the recorded Turbo4 run required 2,504.23 seconds and encountered output truncation and four context-compression events. This difference includes agent orchestration, continuation, file operations, and context management, so it must not be interpreted as a direct measurement of token generation speed.

Agent-task quality Neither cache configuration consistently satisfied the autonomous travel task. Both Qwen runs created all nine required files, but both exceeded the €5,000 budget and contained unreliable final verification. The traditional Qwen run calculated a total of €9,290. The Turbo4 Qwen run reported €3,968.50, although its detailed CSV values totaled approximately €6,947.50.

The traditional Llama run performed substantially worse: it created only one of nine required files and produced a 30-day itinerary limited to Paris, despite requiring eight European countries. The Llama Turbo4 response was more complete, but its reported accommodation and transportation costs already totaled €5,800, exceeding the complete task budget before food, attractions, and other expenses were included.

The apparent quality differences cannot be attributed confidently to KV-cache format. Quantization changes memory representation, while agent quality is also affected by sampling variation, generated output length, tool-call behavior, conversation state, and context compression.

6 Analysis

Your report should include *qualitative evaluation*. That is, try to understand your system (e.g., how it works, when it succeeds and when it fails) by inspecting key characteristics or outputs of your model.

6.1 Context behavior

Both model families experienced pressure on the 65,536-token context limit. Hermes used response continuation and context compression when the conversation and tool history became too large. Turbo4 reduces the physical GPU memory occupied by the KV cache, but it does not increase the logical context limit configured by the server. Consequently, Turbo4 did not prevent output truncation or context compression.

Repeated context compression can also reduce agent reliability because exact requirements, file paths, calculations, and completed steps may be shortened or omitted from the retained summary. This behavior likely contributed to some of the inconsistencies observed in the generated outputs.

6.2 Limitations

The GPU-memory comparison is reasonably controlled because each model pair used the same weights, hardware, context size, and server settings, with the KV-cache format as the intended changed variable. The end-to-end runtime and quality comparisons are less conclusive because most configurations were tested only once, generated-token counts were not matched, tool-call paths differed, context-compression events were inconsistent, and outputs were not verified equally.

Furthermore, `nvidia-smi` measures total GPU allocation rather than the exact amount of KV cache occupied by active tokens. Time to first token, prompt-processing throughput, decode throughput, and exact KV-cache buffer sizes were not measured consistently across all configurations.

7 Conclusion

Summarize the main findings of your project and what you have learned. Highlight your achievements, and note the primary limitations of your work. If you'd like, you can describe avenues for future work.

Turbo4 provided a clear GPU-memory benefit for both evaluated models. It reduced idle allocation by 6.29% for Qwen3.5-9B and 17.27% for Llama 3.1 8B, with Llama saving more than 2.2 GiB. However, the current experiments do not demonstrate a consistent improvement in end-to-end execution time, energy consumption, or autonomous-task quality. The evidence therefore supports Turbo4 primarily as a VRAM-efficiency optimization. Repeated trials with matched token counts, fresh agent sessions, exact KV-cache measurements, time-to-first-token measurements, and decode-throughput measurements are required for a definitive performance comparison.

8 Ethics Statement

What are the ethical challenges and possible societal risks of your project, and what are mitigation strategies?

References

A Appendix (optional)

If you wish, you can include an appendix, which should be part of the main PDF, and does not count towards the 6-8 page limit. Appendices can be useful to supply extra details, examples, figures, results, visualizations, etc. that you couldn't fit into the main paper. However, your grader *does not* have to read your appendix, and you should assume that you will be graded based on the content of the main part of your paper only.